

Deep Research Agent

Agentic RAG 自主深度研究

Agent 开发工程师面试准备

LangGraph + FastAPI + Vue 3
+ ChromaDB + BM25 + SSE

2026-05-16

一、项目概述

基于 Agentic RAG 的自主深度研究系统。Agent 会主动拆解复杂问题、自主选择检索策略、评估结果质量、在质量不达标时自我纠错，最终生成带引用的结构化研究报告。

完整覆盖 AI Agent 核心技术栈：LangGraph 编排、多 Provider LLM、混合检索、检索质量 Critique、查询改写重试、SSE 流式推送、Vue 3 前端。

与传统 RAG 的关键区别

维度	传统 RAG	本系统
检索次数	1次	最多3次（自适应重试）
检索策略	固定向量	Agent自主选择策略
质量判断	无	双维度LLM评分
查询改写	无	3级升级策略
输出	单次回答	结构化报告+引用

二、技术栈全景

以下按架构分层列出项目使用的核心技术，以及每项技术在面试中的讨论要点：

层级	技术	面试要点
Agent编排	LangGraph	显式状态机，非黑盒Agent
LLM	SiliconFlow/Qwen/OpenA	Factory模式多Provider
向量库	ChromaDB	持久化+余弦相似度
关键词	BM25(rank-bm25)	互补语义检索
混合检索	RRF融合(k=60)	向量+BM25重排序
Embed	BGE-large-zh-v1.5	1024维，本地/API双模式
后端	FastAPI+SSE	异步流式，12+端口
前端	Vue3+Vite+TS	CompositionAPI+Pinia

组件库	Naive UI+Tailwind	4页面34源文件
文档	PyMuPDF+docx	PDF/Word/MD/TXT
配置	pydantic-settings	类型安全+热重载

三、Agent 架构详解

LangGraph 状态流转

用户问题

→ Decomposition: LLM拆解2-5个子问题

→ Retrieval: 策略路由+查询改写

→ Critique: 双维度评分(pass/fail)

→ 条件路由:

[PASS] → next or Synthesis

[FAIL]+can retry → 改写重试

[FAIL]+exhausted → 标记低置信度

→ Synthesis: 聚合+报告+引用

四个节点职责

Decomposition:

LLM拆解问题为2-5个子问题，每个标注最优检索策略（semantic/keyword/hybrid），给出选择理由。

Retrieval:

按策略标签路由检索方法。重试时调用QueryRewriter改写查询——L1 broaden（扩展）、L2 switch（切换策略）、L3 rephrase（改写表达），top_k随重试指数扩展。

Critique:

LLM双维度评分——相关性(relevance)+完整性(completeness)→composite_score。≥0.6通过，否则给出retry_suggestion。

Synthesis:

Aggregator去重聚合+冲突检测→ReportGenerator流式生成Mark down→CitationBuilder内联引用标注。

自我纠错机制（核心亮点）

这是与传统RAG最核心的区别。检索不合格时Agent不会接受低质量结果，而是启动3级升级重试：

- L1 broaden: 扩展查询范围，增加同义词
- L2 switch: 切换检索策略，top_k × 2

- L3 rephrase: 完全改写表达， $\text{top_k} \times 4$

3次后仍不达标→标记low_confidence→用最佳可用结果继续→报告中
中标注可信度较低。保证不卡死+信息透明。

四、核心面试问答

Q: 为什么用LangGraph而不是LangChain AgentExecutor ?

LangChain

AgentExecutor是黑盒——无法精确控制每一步。LangGraph

StateGraph显式定义4个节点+1个条件路由，每一步状态完全可控、可观测、可调试。生产环境中当Agent做错误决策时，可以精确追溯到哪个节点、什么状态下出了问题。

Q: 为什么Critique是独立节点？

职责分离。Critique可以用LLM评分（当前）、Cross-Encoder重排序、规则引擎、甚至人工审核——改Critique不影响Retrieval，改Retrieval不影响Critique。每个节点只做一件事，接口清晰，可插拔。

Q: 混合检索vs纯向量检索？

向量检索擅长语义匹配但精确匹配差（数字、专有名词）。BM25互补。例如查询'BERT-base参数量110M'，向量可能返回BERT架构长篇描述，但BM25能精确命中'110M'。RRF融合两者排序，k=60是实验验证的较优参数。

Q: 如何控制Agent幻觉？

三层控制：(1) Critique阈值过滤——质量<0.6触发重试而非强行回答；(2) System Prompt约束'检索不到就说不确定'；(3) 低置信度标记——重试耗尽后在报告中标注可靠性存疑。

Q: SSEvsWebSocket为什么选SSE？

单向推送够用，不需要双向通信。SSE更轻量，浏览器原生EventSource自带自动重连。事件粒度细化到每个子步骤，让用户实时感知Agent思考过程。

Q: 前端为什么从Streamlit迁移到Vue3？

Streamlit的session_state+queue.Queue+st.rerun()轮询模式脆弱，DOM残留频发。Vue3响应式系统天然解决状态同步，EventSource替代轮询，Pinia替代session_state，组件化让每个模块独立可测试。

Q: .env热重载怎么实现？

PATCH→逐行读写.env（不用dotenv_values解析，对特殊字符太脆弱）→正则匹配KEY=VALUE→替换/追加→重建Settings单例→下次LLM调用即生效。API Key掩码后不会回写。

五、前端工程化

Vue 3 + Vite + TypeScript + Naive UI + Tailwind CSS，4个功能页面，34个源文件。

页面	路由	功能
深度研究	/	Agent进度可视化+报告流式渲染
快速检索	/quick-search	对话式UI，秒级响应
资料管理	/documents	上传/预览/删除，自动索引
系统设置	/settings	LLM/嵌入/检索配置，热重载

六、后端API设计

方法	端点	说明
POST	/api/v1/research	提交深度研究
GET	/.../{id}/stream	SSE流式推送
POST	/.../quick-search	快速检索+AI摘要
GET/POST/DEL	/.../documents	资料管理CRUD
GET/PATCH	/.../settings	配置读/写+热重载

SSE事件流

事件	数据	时机
plan_start	{query}	开始拆解
plan_chunk	{index,question,strategy}	每个子问题
retrieval_start	{step,total,strategy}	开始检索
retrieval_result	{count,top_score}	检索完成
critique_result	{score,relevance,passed}	评估完成
synthesis_chunk	{text}	报告流式片段
done	{report_length}	全部完成

七、演示流程（5-8分钟）

1. 空状态 (30s)

展示欢迎页和功能布局

2. 深度研究 (3min)

输入'Transformer相比LSTM有哪些优势？'→Stepper四阶段→左侧拆解的子问题和评分→右侧报告流式生成→来源引用卡片

3. 快速检索 (1min)

侧边栏「快速检索」→输入问题→秒级回复+AI摘要

4. 资料管理 (1min)

「资料管理」→上传PDF→自动索引→预览/删除

5. 系统设置 (30s)

「系统设置」→展示LLM/嵌入/检索配置→说明热重载

八、自我介绍模板（2-3分钟）

我开发了一个基于Agentic

RAG的深度研究Agent，核心价值是让Agent具备主动研究能力。技术上用LangGraph

StateGraph做显式状态编排——定义了拆解、检索、评估、合成四个节点和一个条件路由。检索层是混合检索，向量+BM25+RRF融合，Agent会根据问题特征自主选择检索策略。最核心的是自我纠错机制——检索质量不达标时，自动改写查询、切换策略，最多3次升级重试。后端FastAPI+SSE流式推送，前端Vue

3+TypeScript做了完整的工程化落地——四个功能模块、34个前端文件、12+REST端点。整个项目47次渐进式提交，从设计文档到代码都有完整记录。

九、进阶追问预案

Q: 为什么不直接用LangChain的create_retrieval_chain ?

那是单次检索+回答模式。我的场景需要多步推理——拆解成子问题、每个独立检索评估、失败后重试。create_retrieval_chain做不到这个闭环。

Q: ChromaDB数据量大了怎么办？

ChromaDB适合10万级chunks。更大场景可替换为Milvus——VectorStore是抽象接口，切换只需改一个类。配置里已预留MILVUS_前缀。

Q: 多轮对话怎么设计？

Pinia chat
store维护对话历史，作为research请求的context参数传入后端。SSE事件流天然支持追加。

Q: 如果LLM挂了怎么处理？

所有LLM调用有exception捕获+SSE
error事件。多Provider架构（Factory模式）可通过配置快速切换备用Provider。